

補助予測網による次観測予測 NLL の事後評価 — 何を計算しているか

Belief-Score-Based-Model 実装ノート

2026-09-25

1 目的

補助損失付きの設計 (B: warm start + 観測アンカー + 補助損失, C: cold + 補助損失) は, 粒子集合から次の観測と報酬を予測する補助網を学習時に持つ. 学習ログにはこの補助損失の値を記録していなかったため, 学習済み checkpoint の予測網を学習に使っていないエピソードで評価し, 粒子集合が次観測についてどれだけの情報を持っているかを nats 単位で測る.

2 データ

学習済み checkpoint の方策 (確率的) で, 学習時と別の seed (9000–9019) の 20 エピソードを走らせる. 粒子のノイズと方策のサンプルには, エピソードごとに seed を固定した乱数生成器を渡す. 各ステップで次を記録する.

- 粒子集合 $X_t = \{x_{t,k}\}_{k=1}^K$ ($K = 16$) と実行した行動 a_t
- 次の観測 o_{t+1} と報酬 r_t
- 真の位置 s_t (物差しの計算にだけ使い, 予測網には渡さない)

エピソード最終ステップは次観測がないので観測側の評価から除く.

3 評価量

学習時の補助損失と同じ式を, 未知のデータで評価する. 観測予測網 $h_\phi: (x, a) \mapsto (\mu, \log \sigma)$ を用いて

$$\text{NLL}_{\text{obs}} = \frac{1}{T} \sum_t \left[-\log \frac{1}{K} \sum_{k=1}^K \mathcal{N}(o_{t+1}; \mu(x_{t,k}, a_t), \text{diag } \sigma^2(x_{t,k}, a_t)) \right]. \quad (1)$$

報酬側は o_{t+1} を r_t に, h_ϕ を報酬予測網 h_ψ^r に置き換えた同形の量 NLL_{rew} である. 単位は 1 ステップあたりの nats (観測は 2 次元の合計). 小さいほど予測が当たっている.

4 物差し (同じデータで計算)

Mountain Hike の観測は $o = s + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 3^2 I)$, 遷移は $s' = s + \text{clip}(a) + \eta$, $\eta \sim \mathcal{N}(0, 0.25^2 I)$ である.

理論下限 (oracle). 真の位置 s_t が分かっているときの最良の予測:

$$o_{t+1} \mid s_t, a_t \sim \mathcal{N}(s_t + \text{clip}(a_t), (3^2 + 0.25^2) I) \Rightarrow \text{約 } 5.06 \text{ nats.} \quad (2)$$

どの belief もこれより小さくはならない.

観測のみ (obsonly). 位置の代わりに現在の観測 o_t を使う記憶なしの予測:

$$o_{t+1} | o_t, a_t \sim \mathcal{N}(o_t + \text{clip}(a_t), (2 \cdot 3^2 + 0.25^2) I) \Rightarrow \text{約 } 5.71 \text{ nats.} \quad (3)$$

現在の観測のノイズが加わるので分散がほぼ 2 倍になる.

5 読み方

NLL_{obs} が obsonly に近ければ粒子は観測 1 回分の情報しか持たず, oracle に近ければ真の位置に近い情報を持つ. 両者の差 $\text{obsonly} - \text{oracle} \approx 0.65 \text{ nats}$ のうちどれだけを取れたか,

$$\rho = \frac{\text{obsonly} - \text{NLL}_{\text{obs}}}{\text{obsonly} - \text{oracle}}, \quad (4)$$

を「位置情報の取得率」として読む. これは, $H(o_{t+1} | a_t) - \text{NLL}_{\text{obs}}$ が $I(X_t; o_{t+1} | a_t)$ の変分下界 (Barber–Agakov) であることに基づく. 例として v2 transformer (seed 10) は $\text{NLL}_{\text{obs}} = 5.21$, oracle 5.06, obsonly 5.71 で, $\rho \approx 0.77$ である.

6 結果 (Mountain Hike, seeds 10–14)

各 seed 20 エピソード. oracle は 5.058, obsonly は 5.705 nats (全 run でほぼ一定). ρ の後ろの $\dagger$ は, その seed の 200 update 後の評価 return が -430 以下 (箱外に出続ける退化方策) であることを示す.

設計	エンコーダ	ρ (seed 10 / 11 / 12 / 13 / 14)					NLL_{obs} 平均	ρ 平均
B	Transformer	0.76	0.28 $\dagger$	0.26 $\dagger$	0.00	0.42	5.48 ± 0.16	0.34 ± 0.25
	α -vector	0.55	0.52	0.54	0.50	0.55	5.36 ± 0.01	0.53 ± 0.02
	DeepSets	0.50	0.60	0.07 $\dagger$	0.74	$-0.18\dagger$	5.48 ± 0.23	0.35 ± 0.35
C	Transformer	$-2.62\dagger$	0.22	$-1.02\dagger$	$-0.95\dagger$	0.22	6.24 ± 0.68	-0.83 ± 1.05
	α -vector	0.18	0.22	0.24	0.17	0.26	5.57 ± 0.02	0.21 ± 0.03
	DeepSets	$-1.80\dagger$	0.47	-0.82	$-2.66\dagger$	0.07	6.32 ± 0.75	-0.95 ± 1.16

Table 1: 次観測予測の位置情報取得率 ρ (1 が oracle, 0 が観測のみ, 負は観測のみより悪い予測).

読み.

- 退化方策に落ちていない run では ρ は 0.2–0.76 で, 粒子は観測 1 回より多くの次観測情報を持つ. 最良は B の Transformer seed 10 (0.76), 最も安定しているのは B の α -vector (5 seed と 0.50–0.55).
- B (warm start あり) は C (cold) より一貫して高い (α -vector で 0.53 対 0.21). 補助損失があるとき, warm start は粒子の予測情報を約 2.5 倍にする.
- 退化方策の run では ρ が大きく負になる (最悪 -2.66). エージェントが箱の外へ遠くまで歩き, 観測の値が学習時に稀だった範囲に広がるため, 予測網が過信した誤った予測を出すと考えられる. このとき報酬は -6 で一定なので報酬 NLL はかえって小さくなる (負の値). C の DeepSets seed 12 は退化していないが $\rho = -0.82$ で, 例外として残る.
- 状態復元プローブ (R^2) の順位 (B $\alpha >$ C α) と整合する.

7 注意点

1. 物差しは箱の縁（外側で報酬と挙動が変わる）と行動の上限処理を近似しており， ± 0.02 nats 程度の誤差がある．
2. 予測網を持つのは B / C のみ． A / D と baseline（GRU・RNN・粒子フィルタ）はこの方法では測れず，状態復元プローブ（ R^2 ）で比較する．
3. NLL は予測網の当てはまりの良さも含む． 予測網が不十分なら粒子に情報があっても NLL は下がらないので， ρ は粒子の情報量の下界として読む．
4. 方策ごとに訪れる状態分布が異なる（箱外に出る腕では観測が箱外の値になる）ため，設計間の比較には分布の違いが混ざる．